Kế hoạch chạy và cải tiến trên HSPTEK AIBOX 8550 (KIMQ8550)

Ngày soạn: 10/11/2026. Hạn nộp prototype: 13/11/2026 23:59. Tài liệu này dựa trên bộ tài liệu chip ban tổ chức gửi (thư mục Drive "KIMQ", đã tải về `Downloads/onevoice_chip_docs`), số đo profile trên "QCS8550 (Proxy)" của AI Hub và các lần chạy CPU trên laptop.

Quy ước nhãn: **[TL]** = có trong tài liệu HSPTek (ghi tên tệp); **[Đo]** = số đo của nhóm; **[Ước]** = ước lượng, chưa đo trên hộp thật.

---

0. Tóm tắt một trang

Tài liệu nói gì quan trọng nhất cho ta:

- Hộp chạy Ubuntu 22.04 bản V3.0.0 (hướng dẫn ngày 07/10/2026). Datasheet cũ vẫn ghi 20.04. Lệnh đầu tiên là `cat /etc/os-version`.
- HSPTek cài QAIRT bằng cách chép thư viện vào `/usr/lib` và thư viện DSP **unsigned** v73 vào `/usr/lib/rfsa/adsp`. Nghĩa là hộp chạy được DSP ở chế độ unsigned, đúng cách `qai_appbuilder` cần. Nhưng phiên bản ví dụ là 2.41, còn gói của ta dùng 2.51: phải kiểm xung đột thư viện.
- NPU: HMX hỗ trợ INT8, A16W8 và FP16; bảng thông số ghi **48 TOPS INT8 nhưng chỉ 12 TOPS FP16**. Hiện mọi đồ thị của ta (trừ encoder SenseVoice) đều FP16.
- Bộ nhớ LPDDR5X 4×16 bit. Phép tính trong mục 1.2 cho thấy bước giải mã MT gần như chắc chắn bị giới hạn bởi băng thông bộ nhớ, nên giảm byte trọng số sẽ giảm thời gian bước gần tỉ lệ. **[Ước]**
- Hộp không có micro hay loa tích hợp. Sơ đồ khối có khối "USB Audio" và "HS audio", cần kiểm bằng `arecord -l`.

Đồ nghề đã chuẩn bị (trong gói `s2s_pkg/dist/s2s_edge_pkg`, đã kiểm trên laptop ở mức có thể):

- `tools/runbook.sh` chạy theo giai đoạn A0 đến A9 và AP, ghi log và thời gian từng giai đoạn.
- Chia dev, holdout, quick cố định (`eval/btc_split.json`).
- 3 bản BTC có ồn (babble 10 dB, babble 5 dB, pink 10 dB).
- Công cụ so sánh hai lần chạy có khoảng tin cậy.
- Đo nhiệt khi chạy liên tục, và kiểm micro, loa.

Thời gian trên hộp [Ước]:

Lượt chạy	Thời gian
Kiểm môi trường, cài đặt, kiểm từng đồ thị	15 đến 30 phút
Bộ quick (180 clip)	10 đến 15 phút
Bộ dev (552 clip)	25 đến 40 phút
Toàn bộ 920 clip	45 đến 65 phút

Mỗi thí nghiệm chất lượng trên bộ dev tốn khoảng nửa giờ. Một phiên 4 giờ làm được khoảng 5 đến 6 so sánh.

Thứ tự ưu tiên:

1. Chạy được và đúng trên chip (giống CPU).
2. Chất lượng: ồn, dấu câu, beam.
3. Độ trễ: chế độ HTP, nhiệt, greedy cho câu ngắn.
4. Những việc phải biên dịch lại trên AI Hub (cắt từ vựng, W8A16, nhiều kích thước cửa sổ). Nhóm này chỉ làm khi còn thời gian, vì mỗi vòng biên dịch tốn nhiều giờ.

0.1 Kiểm đúng đầu ra trên chip trước khi có hộp (AI Hub, 10/11)

Chạy inference job trên "QCS8550 (Proxy)" của AI Hub cho cả 9 context binary, với đúng đầu vào golden, rồi so với CPU (`tools/infer_check.py`, kết quả trong `eval/infer_qcs8550*.json`). Profile trước đây chỉ chứng minh mô hình nạp được; lần này chứng minh mô hình tính đúng.

Đồ thị	Kết quả [Đo]	Ghi chú
SenseVoice frontend, encoder	khớp tuyệt đối (cos 1,0; id 100 %)	
NLLB encoder, decoder greedy, decoder beam	khớp tuyệt đối (cos 1,0)	
Zipformer ASR tiếng Việt	logits cos 0,982; **argmax khớp 100 % khung, câu chép giống hệt CPU**	sai số fp16 ở lớp xác suất thấp, không ảnh hưởng giải mã
Piper TTS tiếng Việt	audio cos 0,9995	
MeloTTS tiếng Trung	**thứ tự đầu ra bị đảo** (độ dài trước, audio sau); dài 263 so với 264 đơn vị	ASR của audio chip đọc lại đúng câu
Supertonic TTS Anh, Hàn	dạng sóng lệch (cos 0,21), cùng độ dài, cùng âm lượng	ASR của audio chip đọc lại đúng câu

Lỗi tìm được và đã sửa nhờ lần kiểm này:

1. **MeloTTS đảo thứ tự đầu ra** (lỗi thật). Nếu không sửa, trên hộp mã sẽ lấy con số độ dài làm audio và đầu ra tiếng Trung sẽ câm. TTS giờ chọn đầu ra theo kích thước và kiểu dữ liệu (`s2s_edge/tts.py`, hàm `split_outputs`), đã thử với cả hai thứ tự.
2. **Tiêu chí của `golden.py` sẽ đánh FAIL nhầm 4 đồ thị trên hộp** (ASR tiếng Việt, 3 TTS). Khi đó `session0.sh` sẽ bỏ bộ mô hình QCS8550. Tiêu chí mới: - đầu ra toàn số 0 ở cả hai bên được tính là bằng nhau; - ASR CTC: so argmax từng khung (≥ 99 %); - TTS: dạng sóng khớp, hoặc độ dài lệch ≤ 1 % và âm lượng trong khoảng 0,8 đến 1,25; - đồ thị khác bị đảo đầu ra vẫn FAIL, nhưng in ra cấu hình `"out": [...]` để sửa.

Đã kiểm bằng đối chứng âm: TTS câm, MT bị đảo đầu ra và logits ASR bị xáo đều FAIL đúng. 3. Encoder SenseVoice trước đây **không được `golden.py` kiểm** vì thiếu dữ liệu golden. Đã bổ sung.

Giới hạn: AI Hub dùng máy ảo Android cùng chip, không phải hộp Ubuntu của HSPTek. Lần kiểm này không thay được giai đoạn A2 trên hộp thật.

---

1. Tài liệu chip nói gì

1.1 Truy cập và hệ điều hành

Mục	Nội dung	Nguồn
Hệ điều hành	Ubuntu 22.04 V3.0.0, flash bằng QDL ở chế độ EDL (giữ nút reset khi cắm nguồn 5 đến 7 giây; USB ID `05c6:9008`). **Xóa toàn bộ dữ liệu.**	[TL] `KIMQ8550 Ubuntu 22.04 OS Update Guide`
Mâu thuẫn	Datasheet bảng 1-2 ghi "Android 13, Ubuntu 20.04"; user guide ghi Ubuntu 22.04	[TL] datasheet V1.0.5, user guide trang 4
Tài khoản	`root` / `oelinux123`	[TL] user guide trang 6, OS guide
Cách vào hộp	SSH qua mạng (DHCP, xem IP bằng `ifconfig eth0`); `adb shell` qua cáp Type-C; console nối tiếp qua micro-USB (`minicom -D /dev/ttyACM0 -b 115200`)	[TL] user guide trang 6
Chia sẻ thư mục	`sshfs root@<IP>:/data/ ~/mount` (thư mục `/data` của hộp hiện trên laptop)	[TL] user guide trang 7
Màn hình	Bộ chuyển Type-C sang HDMI	[TL] user guide trang 5
Docker	Cài `docker-ce`; chạy `arm64v8/ubuntu:22.04` với `--privileged` và mount `/usr`, `/lib`, `/dsp`, `/dev`...	[TL] user guide trang 16 và 17
IP tĩnh	Sửa `/etc/dhcpcd.conf`	[TL] user guide trang 18
Bộ cài OS	Thư mục "8. OS Images/Ubuntu/1.0.1" trên Drive không liệt kê được tệp nào (có thể bị giới hạn quyền). Nếu cần flash lại phải xin HSPTek.	tôi kiểm khi tải

Hệ quả cho ta:

- Nếu hộp là 22.04 thì Python 3.10 và glibc 2.35 khớp đúng bánh xe `qai_appbuilder` cp310 manylinux_2_35 có sẵn trong `wheelhouse_cp310`.
- Nếu hộp vẫn là 20.04, có hai lựa chọn: (a) dùng Docker theo đúng cách HSPTek hướng dẫn (`tools/docker_run.sh` của ta làm tương tự); (b) flash lên 22.04. Cách (b) mất dữ liệu và cần bộ cài từ HSPTek.
- Image là bản Yocto dựng theo kiểu Ubuntu (`qti-ubuntu-robotics-image`, toolchain `aarch64-oe-linux-gcc11.2`) theo `KIMQ8550 Ubuntu source code build guide`. Phiên bản glibc thật phải đo bằng `ldd --version` (đã có trong `recon.sh`).

1.2 Phần cứng AI và bộ nhớ

Mục	Nội dung	Nguồn
NPU	Hexagon V73; 4 HVX; 1 HMX với định dạng MAC **INT8, A16W8, FP16**; VTCM lớn; nén băng thông	[TL] datasheet bảng 1-1
Hiệu năng	"Dual eNPU V3, 4 x HVX, HMX, 48 INT8, 12 FP16 TOPs"	[TL] datasheet bảng 1-2
CPU	Bảng 1-2 ghi 1 Prime, 2 Gold, 3 Silver (bản 6 nhân); bản 8C có 1 Prime, 4 Gold, 3 Silver	[TL] datasheet bảng 1-2, 4-1
Bộ nhớ	LPDDR5X 4 kênh × 16 bit, "thiết kế cho 4266 MHz"	[TL] datasheet bảng 1-1
RAM	Bảng 1-2 ghi uMCP LPDDR5 12 GB; user guide ghi 16 GB	[TL] mâu thuẫn, đo bằng `free -h`

Mục	Nội dung	Nguồn
Lưu trữ	UFS 128 GB	[TL]
Công suất	Tối đa 18 W, thường 11,9 W, tối thiểu 5,1 W (Ubuntu 20.04)	[TL] datasheet 3.15
Nhiệt	"TBD"	[TL] datasheet 3.16

Băng thông và bước giải mã MT. [Ước] Bus 64 bit (8 byte). Nếu 4266 MHz là xung nhịp (8533 MT/s) thì đỉnh khoảng 68 GB/s; nếu là MT/s thì khoảng 34 GB/s. Decoder MT (`mt_dec`, 926 MB) chạy 15,23 ms mỗi bước trên QCS8550 **[Đo]**. Nếu mỗi bước đọc toàn bộ trọng số thì cần khoảng 61 GB/s. Con số này chỉ khớp với cách hiểu 68 GB/s, và rất gần đỉnh. Kết luận: bước giải mã gần như chắc chắn bị giới hạn bởi băng thông. Cắt từ vựng và trọng số int8 sẽ rút ngắn bước gần tỉ lệ với số byte giảm được. Đây là suy luận; tính năng nén băng thông của chip có thể làm số liệu lệch. Xác nhận bằng profile sau khi có đồ thị nhỏ hơn.

FP16 so với INT8. Bảng thông số ghi 12 so với 48 TOPS. Với đồ thị bị giới hạn bởi tính toán (encoder ASR cửa sổ 15 s, TTS), chuyển sang lượng tử hóa có thể nhanh hơn nhiều. Datasheet không ghi thông lượng của A16W8. Zipformer w8a16 đã từng cho chất lượng kém **[Đo]**, nên đây là hướng có rủi ro chất lượng (mục 3.3, L6).

1.3 Âm thanh

- Module có 2 SoundWire (một cho codec, một cho ampli), 4 giao diện DMIC, 1 I2S **[TL]** datasheet bảng 1-2 và 2-8. Đây là chân trên module, chưa chắc carrier board có nối ra micro hay loa.
- Sơ đồ khối carrier (`HSP046-Blockdiagram`) có khối "USB Audio", "HS Audio signal" và hub USB 3.0 với 4 cổng Type-A **[TL]**. Có thể có cổng tai nghe; phải kiểm bằng `arecord -l` và `aplay -l`.
- Phương án chắc chắn: micro và loa USB (hoặc tai nghe USB) cắm vào cổng Type-A. Dùng `tools/audio_check.sh` để kiểm mức tín hiệu.
- Thiết bị USB thường **không** phải thiết bị mặc định của ALSA. Chế độ live giờ nhận `S2S_MIC` và `S2S_SPK`, ví dụ `S2S_MIC=plughw:1,0 S2S_SPK=plughw:1,0 bash run_demo.sh vi en --continuous`.

1.4 QAIRT trên hộp

- User guide trang 10 và 11 **[TL]**: tải QAIRT (ví dụ 2.41.0.251128), rồi `scp` các tệp `lib/aarch64-oe-linux-gcc11.2/*` vào `/usr/lib/`, `lib/hexagon-v73/unsigned/*` vào `/usr/lib/rfsa/adsp/`, `bin/...` vào `/usr/bin/`. Kiểm bằng `snpe-platform-validator --runtime dsp` và `qnn-platform-validator --backend dsp --testBackend`.
- **Tin tốt:** DSP chạy được thư viện unsigned, nên `qai_appbuilder` (cũng dùng skel unsigned) có khả năng chạy được.
- **Rủi ro:** nếu `/usr/lib` đã có `libQnnHtp.so` bản 2.41, trình nạp có thể lấy nhầm bản đó thay vì bản 2.51 trong bánh xe. Context binary của ta gắn với QAIRT 2.51, nên sẽ báo lỗi khi nạp. `recon.sh` giờ in phiên bản các thư viện này.
- **Đã kiểm trong mã nguồn bánh xe** (`qnncontext.py`): thư viện backend được nạp theo đường dẫn tuyệt đối trong thư mục `libs` của bánh xe (tốt), nhưng `ADSP_LIBRARY_PATH` **chỉ được đặt khi đang trống**. Nếu hộp đã đặt sẵn biến này trỏ vào skel 2.41 thì sẽ lệch phiên bản.
- **Cách xử lý (đã làm):** `tools/env.sh` đặt `LD_LIBRARY_PATH` (trước khi Python chạy) và `ADSP_LIBRARY_PATH` sao cho thư viện của bánh xe đứng trước, thư mục hệ thống đứng sau. `backends.py` cũng tự đặt `ADSP_LIBRARY_PATH` như vậy. `runbook.sh`, `session0.sh`, `run_btc.sh`, `run_demo.sh` đều đã gọi `env.sh`. Muốn dùng QAIRT của hệ thống thì đặt `S2S_SYSTEM_QNN=1`. Nếu vẫn lỗi, phương án dự phòng là bản `.dlc` (nạp được với QAIRT 2.41 trở lên). Bản này để ngoài gói (7,9 GB, `s2s_pkg/models_qcs8550`), phải mang theo trên ổ ngoài.

1.5 Những gì tôi chưa xem

- Video training (MP4, không tải).
- Bản vẽ cơ khí, schematic (không liên quan đến phần mềm).
- Mã mẫu `hsp-aidemo-2.zip` (nhận diện khuôn mặt bằng C) và `yolonas-demo.zip`: đã tải nhưng chưa mở; đây là ví dụ thị giác dùng GStreamer với `qtimlsnpe`, không dùng cho âm thanh.
- Tài liệu Qualcomm DCN trong `Qualcomm_DCN.txt` chỉ là danh sách mã tài liệu cho QCS6490.

---

2. PHẦN A. Chạy trên phần cứng với dữ liệu BTC

2.1 Mang gì đến hộp

Thứ	Ở đâu	Dung lượng
Gói chạy (mô hình QCS8550 và IQ-9075, bánh xe cp312 và cp310, công cụ)	`Downloads/s2s_pkg/dist/s2s_edge_pkg`	8,2 GB
Dữ liệu mẫu BTC	`Downloads/onevoice_btc_data`	143 MB
3 bản BTC có ồn	`Downloads/onevoice_btc_noisy`	3 × 98 MB
Dự phòng DLC (nếu context binary 2.51 không nạp được)	`Downloads/s2s_pkg/models_qcs8550`	7,9 GB
Phần cứng	cáp mạng, cáp Type-C có dữ liệu, cáp micro-USB (console), bộ chuyển Type-C sang HDMI, micro và loa USB (hoặc tai nghe USB), quạt nhỏ nếu hộp nóng	

Chép lên hộp qua mạng: `scp -r s2s_edge_pkg root@<IP>:/data/` (hoặc dùng sshfs như mục 1.1). Thư mục `/data` là nơi HSPTek dùng trong mọi ví dụ.

2.2 Các giai đoạn

Lệnh chung, chạy trong thư mục gói trên hộp:

```
DATA=/data/onevoice_btc_data bash tools/runbook.sh A0 A1 A2 A3 A4 A5 AP A6
```

Mỗi giai đoạn ghi log riêng, thời gian vào `timing.tsv`, và cuối cùng đóng gói `logs/runbook_<giờ>_<tag>.tgz` để chép về.

Giai đoạn	Làm gì	Thời gian [Ước]	Đạt khi	Nếu không đạt
A0 recon	OS, glibc, Python, QAIRT có sẵn, cdsp, nhiệt, âm thanh	2 phút	có `/etc/os-version`, `cdsp running`, glibc ≥ 2.35	glibc thấp hoặc Python 3.8: dùng `tools/docker_run.sh`
A1 install	cài offline từ `wheelhouse` hoặc `wheelhouse_cp310`, import `qai_appbuilder`	5 đến 15 phút	import được, in ra thư mục `libs`	xem mục 1.4 (xung đột 2.41)

Giai đoạn	Làm gì	Thời gian [Ước]	Đạt khi	Nếu không đạt
A2 golden	nạp từng đồ thị trên NPU, so với đầu ra CPU (cos ≥ 0,99)	2 đến 5 phút	`all pass`; in thời gian nạp từng đồ thị	đồ thị nào FAIL thì thử bản DLC của riêng đồ thị đó
A3 perf	lặp lại A2 với BURST, HIGH_PERFORMANCE, NONE	5 đến 10 phút	biết chế độ nào nhanh nhất	giữ BURST
A4 soak	chạy `mt_decB` và `tts_zh` liên tục 10 phút mỗi cái	20 phút	chậm đi dưới 10 % giữa phút đầu và phút cuối	thêm quạt, nghỉ giữa các chiều, hoặc thử HIGH_PERFORMANCE (bánh xe `qai_appbuilder` 2.51.40 chỉ có DEFAULT, HIGH_PERFORMANCE, BURST)
A5 smoke	2 clip mỗi chiều × 6 chiều	3 đến 5 phút	có audio đầu ra, không lỗi	đọc log chiều bị lỗi
AL live	chạy đúng đường mã live (có warmup), thay micro bằng file WAV	2 đến 4 phút	PASS; in thời gian nạp từng mô hình	đọc log; kiểm `env.sh`
AP parity	30 clip đầu mỗi chiều (đúng các clip CPU đã chạy), so với `eval/cpu_ref`	10 đến 15 phút	không chiều nào "B worse" có ý nghĩa	tìm đồ thị lệch số (fp16 trên chip)
A6 quick	bộ quick 180 clip	10 đến 15 phút	bảng chất lượng và độ trễ	
A7 dev	bộ dev 552 clip: mốc cơ sở để so mọi phương pháp	25 đến 40 phút		
A8 holdout	bộ holdout 368 clip: **chỉ chạy một lần** với cấu hình cuối	17 đến 29 phút		
A9 noisy	bộ quick trên mỗi bản có ồn	10 đến 15 phút mỗi bản	biết chất lượng tụt bao nhiêu theo mức ồn	

Ngoài runbook:

```
bash tools/audio_check.sh                       # liệt kê thiết bị thu và phát
bash tools/audio_check.sh hw:1,0 hw:1,0         # thu 5 s, in mức dBFS, phát lại
bash tools/audio_check.sh hw:1,0 - 120          # thu 2 phút tiếng phòng làm việc ->
logs/office_ambience.wav
python tools/mix_noise.py --data /data/onevoice_btc_data --out /data/btc_office5 --kind file --noise
logs/office_ambience.wav --snr 5
```

Bản ồn thu từ chính micro trong phòng thật là bản sát với điều kiện "doanh nghiệp, ồn ào" nhất mà ta có.

2.3 Cách tính thời gian

Số đo profile trên QCS8550 **[Đo]** (`models_qcs8550/profile.json`, mỗi lần gọi):

Đồ thị	ms	Ghi chú
ASR vi (Zipformer, cửa sổ 15 s)	163	
ASR en/zh/ko (SenseVoice frontend + encoder)	58 + 237	encoder w8a16

Đồ thị	ms	Ghi chú
MT encoder	7,5	
MT decoder greedy, một bước	15,2	
MT decoder beam 4, một bước	30,8	
TTS vi (Piper), một cửa sổ	520	
TTS zh (MeloTTS), một cửa sổ	741	
TTS en/ko (Supertonic), một cửa sổ	701	

Số bước giải mã **[Đo]** bằng tokenizer NLLB trên câu đích chuẩn của BTC: trung vị 8 (đích tiếng Hàn) đến 21 (đích tiếng Việt từ tiếng Anh), p90 12 đến 35. Số cửa sổ TTS: giả định 1 đến 2 **[Ước]**.

- Pipeline một clip (beam): ASR 0,16 đến 0,30 s, cộng MT 0,25 đến 1,1 s, cộng TTS 0,5 đến 1,5 s, tổng **khoảng 1,0 đến 2,9 s**, chưa kể chi phí phía CPU.
- Khi đánh giá còn cộng thêm dịch câu gốc chuẩn (`mt_gold`, thêm 0,6 đến 0,9 s) và ASR audio đầu ra ("heard", thêm 0,16 đến 0,6 s), tổng **khoảng 2,1 đến 4,1 s mỗi clip**. Dùng `EXTRA="--no_gold"` bớt được khoảng một phần tư.
- Thời gian nạp mô hình mỗi chiều (khoảng 2,2 GB context binary) là ẩn số lớn nhất. A2 giờ in ra thời gian này.

Bộ	Số clip	Thời gian [Ước] (chưa tính nạp)
quick	180	6 đến 12 phút
dev	552	19 đến 38 phút
holdout	368	13 đến 25 phút
toàn bộ	920	32 đến 63 phút

So sánh với laptop [Đo]: pipeline đầy đủ trên CPU laptop (ONNX Runtime) tốn 47 đến 99 giây mỗi clip (`logs/btc_cpu30*.json`). Bộ quick trên laptop mất 2,4 đến 5 giờ. Vì vậy laptop chỉ dùng cho thí nghiệm mức văn bản (MT với văn bản chuẩn), không dùng cho thí nghiệm âm thanh đầy đủ.

Vòng AI Hub [Ước]: link 10 đến 30 phút mỗi đồ thị (lần relink 09/11). Biên dịch đồ thị lớn và profile: chưa đo kỹ, ước 20 đến 60 phút mỗi đồ thị, chạy song song được. Một thay đổi cần biên dịch lại tốn **nửa ngày** tính cả kiểm tra.

2.4 Lịch phiên đề xuất

Phiên 1 (khoảng 3 đến 4 giờ): chạy được và đúng. A0, A1, A2, A3, A4, A5, AP, A6, kiểm micro và loa, thu tiếng phòng. Kết quả: biết hộp chạy được không, chip có giống CPU không, chất lượng và độ trễ thật trên bộ quick.

Phiên 2 (khoảng 4 giờ): mốc và chất lượng. A7 (mốc dev), rồi các thí nghiệm Q1 đến Q4 ở Phần B (mỗi cái khoảng 30 phút trên dev, hoặc 10 đến 15 phút trên quick để sàng lọc trước).

Phiên 3: độ trễ và chốt. L1 đến L3, quay video demo với mạng tắt, rồi chạy A8 holdout **một lần** với cấu hình cuối và lưu log thô.

Hạn 13/11 rất gần. Nếu chỉ có một phiên, làm Phiên 1 và quay video demo.

---

3. PHẦN B. Cải tiến trong lúc chạy

3.1 Quy trình thử nghiệm (bắt buộc)

1. **Một thay đổi mỗi lần**, đặt `TAG` cho lần chạy, ví dụ `TAG=greedy MODELS=models_qcs8550/models_fast.json bash tools/runbook.sh A6`.
2. **Sàng lọc trên quick** (10 đến 15 phút), **quyết định trên dev** (25 đến 40 phút), **xác nhận một lần trên holdout** ở cuối.
3. **So bằng thống kê:** `python tools/compare_runs.py <mốc>/btc_vi-ko.json <biến thể>/btc_vi-ko.json --metric heard`. Chỉ nhận thay đổi khi khoảng tin cậy 95 % nằm hẳn trên 0 (với `src_err` thì nằm hẳn dưới 0).
4. **Độ nhạy:** bộ dev có 60 đến 108 clip mỗi chiều. Thử trên laptop với n = 30, một thay đổi làm mất 3,8 chrF có khoảng tin cậy [-8,0; -1,2]. Nên chỉ những thay đổi khoảng 1,5 đến 2 chrF trở lên mới thấy được trên dev. Thay đổi nhỏ hơn cần cộng gộp nhiều chiều.
5. **Không chỉnh theo holdout.** Không dùng clip BTC để huấn luyện hay viết luật.
6. **Lưu ý:** luật khôi phục "?" (`vi_punct`) đã được chỉnh trên **toàn bộ** BTC từ trước khi có bộ chia. Vì vậy holdout **không sạch** với riêng luật này; kết quả Q2 chỉ cho biết luật không làm hại, không chứng minh nó tổng quát. Các phương pháp khác chưa đụng đến BTC nên holdout vẫn sạch với chúng.

Độ đo dùng để quyết định: **"heard" chrF** (ASR của audio đầu ra, gần với cách chấm điểm nhất), kèm `mt_e2e` chrF (văn bản dịch) và `src_err` (lỗi ASR nguồn) để biết khâu nào thay đổi.

3.2 Chất lượng trước (trọng số 45 %)

#	Thí nghiệm	Vì sao	Cách chạy trên hộp	Thời gian	Quyết định
Q0	Chip giống CPU (AP)	fp16 trên HTP có thể lệch số; mọi thứ khác vô nghĩa nếu chip sai	`runbook.sh AP`	10 đến 15 phút	không chiều nào tụt có ý nghĩa
Q1	Beam 4 so với greedy	beam tốn gấp đôi thời gian giải mã; cần biết nó đáng bao nhiêu chrF	`TAG=greedy MODELS=models_qcs8550/models_fast.json runbook.sh A7`	25 đến 40 phút	giữ beam nếu hơn có ý nghĩa ở chiều yếu (vi-ko, vi-zh)
Q2	Dấu câu và viết hoa cho ASR tiếng Việt: mô hình (mặc định) so với luật so với tắt	mô hình vibert-capu int8 thắng luật trên laptop (bảng dưới); cần xác nhận trên chip và bộ dev	`TAG=punct_rule S2S_VI_PUNCT=1 runbook.sh A7`, `TAG=punct_off S2S_VI_PUNCT=0 runbook.sh A7`	25 đến 40 phút mỗi biến thể (chỉ các chiều vi→x)	giữ mô hình nếu không thua luật có ý nghĩa
Q3	Chịu ồn	điều kiện thi là văn phòng ồn; BTC sạch	`NOISY="... btc_babble10 btc_babble5 btc_pink10 btc_office5" runbook.sh A9`	10 đến 15 phút mỗi bản	biết mức tụt; nếu tụt nhiều, thử Q4

#	Thí nghiệm	Vì sao	Cách chạy trên hộp	Thời gian	Quyết định
Q4	Lọc ồn GTCRN và "trộn lại tín hiệu gốc"	lọc ồn rời không giúp (đo trước đây); bài Iwamoto (2022) cho thấy cộng lại một phần tín hiệu gốc có thể giúp	cần thêm tham số vào mã (chưa làm), so trên `btc_babble5`	15 phút mỗi hệ số	chỉ giữ khi có ý nghĩa
Q5	Glossary thuật ngữ	từ ngữ doanh nghiệp, tên riêng	`EXTRA="--glossary glossary.json"`	10 đến 15 phút	giữ nếu không làm tụt chiều nào
Q6	Micro thật: mức thu, khoảng cách	độ ồn và mức tín hiệu thật quyết định ASR	`audio_check.sh`, chỉnh `alsamixer`, nói thử 10 câu	20 phút	đỉnh tiếng nói khoảng -12 đến -3 dBFS
Q7	VAD khi chạy trực tiếp	VAD năng lượng dễ cắt sai khi ồn	chạy chế độ trực tiếp trong phòng; ghi số câu bị cắt sai	20 phút	chưa có công cụ tự động

Kết quả trên laptop cho Q2 (11/11). Mô hình `dragonSwing/vibert-capu` (viBERT-base, CC-BY-SA-4.0) xuất sang ONNX int8 (111 MiB), viết lại phần trước và sau mô hình bằng numpy (`s2s_edge/punct_vi.py`, khớp 100 % với mã gốc trên 588 câu, `tools/check_punct_vi.py`). chrF của NLLB:

Đầu vào MT	vi→en	vi→ko	vi→zh
280 câu chuẩn bỏ dấu: luật / mô hình / câu gốc	50,4 / **52,0** / 52,3	25,9 / **28,9** / 31,3	
Chữ ASR thật, 30 clip: luật / mô hình	**55,6** / 55,1	25,6 / **34,1**	31,0 / **36,3**

Thời gian: 12 ms mỗi câu (ONNX int8) trên CPU laptop với 4 luồng, 26 ms trên chữ ASR thật (có câu chạy 3 lượt). Trên CPU ARM của hộp chưa đo. Mô hình còn đổi số đọc thành chữ số ("bảy giờ" → "7 giờ") và đôi khi sai ("ba anh ta" → "3. Anh ta").

Bộ mô hình MT thứ hai: NLLB-1.3B (11/11) — đã loại, chuyển ra `s2s_pkg/archive_nllb13/`. Đo trên toàn bộ 920 câu BTC (văn bản chuẩn, kiểm thống kê): trung bình 6 chiều A (600M beam 4) 44,0 so với B (1.3B beam 2) 43,7; B kém có ý nghĩa ở en→vi (−2,7) và vi→ko (−3,8), chỉ nhỉnh không có ý nghĩa ở ko→vi, vi↔zh (+1,1 đến +1,6). Trên chữ ASR thật vi→ko: 34,1 so với 27,9. Kết quả 60 câu bên dưới đã bị phóng đại do mẫu nhỏ. Giữ 600M. Cơ chế tự hạ cấp beam (`MtFallback`) vẫn dùng cho 600M.

	600M beam 4 (mặc định)	1.3B beam 2 (`models_qcs8550_13b`)
chrF, 60 câu chuẩn: vi→ko / ko→vi / vi→zh / zh→vi	**32,1** / 39,3 / 33,8 / 49,2	29,8 / **41,5** / **38,3** / **53,2**
Thời gian trên chip (AI Hub, QCS8550)	enc 7,5 ms; bước beam 30,8 ms	enc 23,5 ms; bước greedy 33,6 ms; bước beam 2 **chưa đo** (profile lỗi `SET_PROFILE`, inference chạy đúng)
Dung lượng MT	2,4 GB	encoder 1,3 GB + decoder beam 2 1,6 GB (thêm decoder greedy, beam 4 dự phòng)
Kiểm đúng trên chip (`infer_check`)	PASS	PASS sau khi đặt lại thứ tự đầu ra (`"out"` trong `models.json`)

- Beam 4 của 1.3B lỗi thiếu bộ nhớ (`MEM_ALLOC`) trên máy ảo AI Hub; vẫn để trong `models_beam.json` (golden riêng `golden_13b_b4`) để thử một lần trên hộp thật.
- 1.3B greedy **thua** 600M beam 4 (vi→ko 26,9 so với 32,1), nên đừng dùng `models_fast.json` của bộ 1.3B.
- Trên hộp: `MODELS=models_qcs8550_13b/models.json bash tools/runbook.sh A2 AL A6`, rồi `compare_runs.py` giữa hai thư mục kết quả. Chọn theo "heard" chrF và độ trễ thật. Ước tính mỗi câu chậm hơn khoảng 0,2 đến 0,4 s, chưa đo.

Những việc làm được ngay trên laptop trong lúc chờ hộp: chạy `mt_gold` với các biến thể dịch (bản HF tham chiếu trên GPU), soạn glossary miền doanh nghiệp, đếm phân bố độ dài clip.

3.3 Độ trễ sau (trọng số 20 %)

#	Thí nghiệm	Vì sao	Cách chạy	Thời gian	Rủi ro chất lượng
L1	Chế độ HTP	BURST có thể bị hạ xung khi nóng	A3	10 phút	không
L2	Nhiệt	thông số nhiệt là "TBD"	A4; nếu chậm đi trên 10 % thì thêm quạt	20 phút	không
L3	Greedy cho câu ngắn, beam cho câu dài	greedy rẻ gấp đôi; câu ngắn ít lợi từ beam	cần thêm mã chọn theo độ dài (chưa làm), so trên dev	40 phút	thấp, đo bằng Q1
L4	Thời gian nạp mô hình	nạp chậm làm câu đầu tiên rất chậm	đọc thời gian nạp ở A2; nạp sẵn tất cả đồ thị khi khởi động nếu RAM đủ (khoảng 4 GB cho cả 6 chiều)	10 phút	không
L5	Cắt từ vựng đầu ra theo ngôn ngữ đích	embedding chiếm khoảng 31 % decoder; bước giải mã bị giới hạn bởi băng thông (mục 1.2)	biên dịch lại trên AI Hub	nửa ngày	thấp nếu kiểm parity
L6	W8A16 cho NLLB, rồi cho TTS	FP16 12 TOPS so với INT8 48; trọng số 8 bit giảm byte đọc	AIMET trên laptop, rồi biên dịch AI Hub	1 đến 2 ngày	trung bình (Zipformer w8a16 từng kém)
L7	Nhiều kích thước cửa sổ ASR, TTS	câu ngắn đang trả giá cho cửa sổ đầy	biên dịch lại	nửa ngày đến 1 ngày	thấp

L5 đến L7 cần AI Hub và nhiều giờ, nên **chỉ làm sau khi Phiên 1 đã chứng minh chip chạy đúng**. Chi tiết kỹ thuật và tài liệu tham khảo của từng mục ở `docs/improvement_playbook.pdf`.

---

4. Đồ nghề đã chuẩn bị

Tệp	Làm gì	Đã kiểm
`tools/runbook.sh`	chạy theo giai đoạn, ghi log, thời gian, đóng gói	cú pháp (`bash -n`); chưa chạy trên hộp
`tools/recon.sh`	thêm: `/etc/os-version`, phiên bản QAIRT có sẵn, cdsp, nhiệt, governor	cú pháp

Tệp	Làm gì	Đã kiểm
`tools/btc_split.py`, `eval/btc_split.json`	chia dev, holdout, quick cố định theo id; vi-ko và ko-vi chung một phía	chạy trên laptop
`tools/eval_btc.py`	thêm `--split_file`, `--split`, `--no_gold`	chạy 1 clip trên CPU, id đúng phía holdout
`tools/compare_runs.py`	bootstrap ghép cặp, khoảng tin cậy 95 %	trên laptop: cùng tệp cho chênh 0; biến thể bị làm hỏng cho "B worse"
`tools/mix_noise.py`	tạo bản BTC có ồn (babble, pink, bản thu thật)	đo lại SNR: đúng 5,0 và 10,0 dB
`tools/thermal_soak.py`	chạy một đồ thị liên tục, ghi độ trễ và nhiệt	chạy 15 s trên CPU (laptop không đọc được nhiệt)
`tools/audio_check.sh`	liệt kê, thu, đo mức, phát lại; thu tiếng phòng	cú pháp; cần hộp thật
`tools/golden.py`	thêm thời gian nạp từng đồ thị	chạy trên CPU
`s2s_edge/backends.py`	`S2S_PERF` chọn chế độ HTP	cú pháp; đường NPU chưa chạy
`s2s_edge/conversation.py`	`S2S_VI_PUNCT=0` tắt khôi phục "?"	import được
`eval/cpu_ref/`	kết quả CPU 30 clip mỗi chiều để so parity	có sẵn
`tools/env.sh`	đưa thư viện QAIRT 2.51 của bánh xe lên trước QAIRT 2.41 có sẵn trên hộp	cú pháp
`tools/infer_check.py`	kiểm đầu ra trên chip ảo của AI Hub	đã chạy: 9 trên 9 đồ thị đạt (mục 0.1)
`tools/live_selftest.py`	chạy chế độ live thật, thay micro bằng WAV	PASS trên CPU: vi→en và hội thoại vi,ko
`s2s_edge/registry.py`, `__main__.py`	warmup: nạp và chạy mỗi mô hình một lần trước câu đầu tiên (`--no-warmup` để bỏ)	đã chạy trên CPU
`s2s_edge/audio.py`	`S2S_MIC`, `S2S_SPK` chọn thiết bị ALSA	import và tham số đúng
`s2s_edge/tts.py`	chọn đầu ra TTS theo kích thước và kiểu, không theo vị trí	đã thử cả thứ tự CPU và thứ tự chip
`glossary_enterprise.json`	bản nháp 34 thuật ngữ văn phòng (68 khóa có biến thể chữ hoa): cách đọc cho Piper, dạng chữ cho zh/ko	kiểm ở mức văn bản; chưa nghe audio; phải qua Q5
`run_demo.sh`	sửa lỗi trỏ vào mô hình IQ-9075; giờ dùng bộ QCS8550	cú pháp
`Downloads/onevoice_btc_noisy/`	3 bản ồn, mỗi bản 920 tệp	tạo xong

Mọi tệp đã chép vào gói với kết thúc dòng kiểu Linux.

---

5. Rủi ro và câu hỏi mở

1. **Phiên bản OS thật của hộp** (20.04 hay 22.04)? Quyết định đường cài đặt.
2. **Xung đột QAIRT 2.41 trên hộp và 2.51 trong bánh xe.** Chưa thể kiểm trước khi có hộp.
3. **RAM 12 hay 16 GB.** Ảnh hưởng việc nạp sẵn cả 6 chiều (khoảng 4 GB mô hình cộng bộ nhớ chạy).
4. **Âm thanh:** carrier có cổng tai nghe dùng được không, hay phải mang thiết bị USB.

5. **Nhiệt:** chưa có thông số. Một lượt chạy 920 clip giữ NPU bận khoảng một giờ.
6. **Đường NPU của mã chưa từng chạy trên thiết bị thật.** Mọi ước lượng thời gian ở trên dựa trên profile của AI Hub (dữ liệu ngẫu nhiên) cộng giả định số bước và số cửa sổ.
7. **Cách ban tổ chức đo độ trễ** (đến âm đầu tiên hay hết câu) vẫn chưa rõ.